

Ders1

```
//import javax.swing.; // 1. İçe Aktarma /** *
```

- @author ummualtin

```
*/ public class Ders1 { // 2. Sınıf Tanımı
```

```
public static void main(String[] args) { // 3. Ana Metot (Main Method)
```

```
    JFrame fr = new JFrame(); // 4. JFrame Nesnesi Oluşturma  
    // JFrame sınıfından bir nesne (örnek) oluşturuluyor.  
    // JFrame, masaüstü uygulamalarında bir pencereyi temsil eder.  
    // 'fr' bu pencere nesnesine verdiğimiz isim (referans).
```

```
    JButton but = new JButton("click"); // 5. JButton Nesnesi Oluşturma  
    // JButton sınıfından "but" adında bir nesne oluşturuluyor.  
    // JButton, üzerine tıklanabilen bir düğmeyi temsil eder.  
    // Parantez içindeki "click" metni, düğmenin üzerinde görünecek yazıdır.
```

```
    but.setBounds(100, 90, 100, 50); // 6. Düğmenin Konumunu ve Boyutunu Ayarlama  
    // 'but' düğmesinin pencere ('fr') içindeki yerini ve boyutlarını belirliyoruz.  
    // setBounds(x, y, genişlik, yükseklik) şeklinde çalışır:  
    // x = 100: Düğmenin sol üst köşesinin pencerenin sol kenarından uzaklığı (piksel).  
    // y = 90: Düğmenin sol üst köşesinin pencerenin üst kenarından uzaklığı (piksel).  
    // genişlik = 100: Düğmenin genişliği (piksel).  
    // yükseklik = 50: Düğmenin yüksekliği (piksel).  
    // Bu komutun düzgün çalışması için pencerenin layout manager'ının null olması gerekir (bkz. adım 8).
```

```
    fr.add(but); // 7. Düğmeyi Pencereye Ekleme  
    // Oluşturduğumuz 'but' düğmesini, 'fr' penceresine ekliyoruz.  
    // Artık düğme pencerenin bir parçasıdır.
```

```
    fr.setSize(300, 300); // 8. Pencerenin Boyutunu Ayarlama
```

```
//300 piksel yatay(x) , 300 piksel dikey(y) şeklinde ayarlanır

fr.setLayout(null); // 9. Layout Manager'ı Devre Dışı Bırakma
// Pencerenin ('fr') layout manager'ını (yerleşim yöneticisini)
null olarak ayarlıyoruz.
// Layout manager'lar, bileşenlerin (düğme, metin alanı vb.)
pencere içinde nasıl
// düzenleneceğini otomatik olarak ayarlar. 'null' yapmak, bu
otomatik düzenlemeyi
// kapatır ve bileşenleri 'setBounds' gibi komutlarla manuel
olarak yerleştirmemize
// olanak tanır. Basit örnekler için kolay olsa da, karmaşık
arayüzlerde ve
// farklı ekran boyutlarına uyum sağlamada sorun yaratabilir.

fr.setVisible(true); // 10. Pencereyi Görünür Yapma
// Oluşturulan 'fr' penceresini ve içindeki bileşenleri (düğme)
ekranda görünür hale getiriyoruz.
// Varsayılan olarak, oluşturulan pencere görünmezdir. Bu
komutla görünür yapılır.
}
}
```

Özetle:

Bu kod parçası, Java'nın Swing kütüphanesini kullanarak basit bir grafiksel kullanıcı arayüzü (GUI) oluşturur. 300x300 piksel boyutunda bir pencere yaratır, içine "click" yazan bir düğme ekler, düğmenin konumunu ve boyutunu manuel olarak ayarlar ve son olarak pencereyi ekranda gösterir.

Ders2

```
import java.awt.event.*; // 1. Olay (Event) İşleme Paketini İçer Aktarma

// Fare tıklamaları, klavye girişleri gibi olayları yönetmek için gerekli sınıfları içerir.
// Özellikle ActionListener ve(ActionEvent bu örnekte kullanılır.

import javax.swing.*; // 2. Swing GUI Paketini İçer Aktarma

// JFrame, JButton, JTextField gibi grafiksel arayüz bileşenlerini içerir.
```

```
public class Ders2 {  
  
    public static void main(String[] args) {  
  
        JFrame f = new JFrame("Button Example"); // 5. JFrame Nesnesi Oluşturma (Başlıklı)  
        // Pencere nesnesi 'f' oluşturuluyor.  
        // "Button Example" metni pencerenin başlık çubuğunda görünecek.  
  
        final JTextField tf = new JTextField(); // 6. JTextField Nesnesi Oluşturma  
        // Tek satırlık metin girişi veya gösterimi için kullanılan metin alanı nesnesi 'tf'  
        oluşturuluyor.  
        // 'final' anahtar kelimesi: Bu değişkenin ('tf') referansının daha sonra  
        değiştirilemeyeceği anlamına gelir.  
        // Bu, özellikle lambda ifadesi (adım 10'daki gibi) içinden erişilen yerel değişkenler  
        için gereklidir.  
  
        tf.setBounds(45, 75, 220, 20); // 7. Metin Alanının Konumunu ve Boyutunu Ayarlama  
        // 'tf' metin alanının pencere ('f') içindeki yeri ve boyutları ayarlanıyor.  
        // x=45, y=75, genişlik=220, yükseklik=20 piksel.  
  
        JButton b = new JButton("Click Here"); // 8. JButton Nesnesi Oluşturma  
        // Üzerinde "Click Here" yazan bir düğme ('b') oluşturuluyor.  
  
        b.setBounds(100, 100, 100, 50); // 9. Düğmenin Konumunu ve Boyutunu Ayarlama  
        // 'b' düğmesinin pencere ('f') içindeki yeri ve boyutları ayarlanıyor.  
        // x=100, y=100, genişlik=100, yükseklik=50 piksel.  
  
        b.addActionListener((ActionEvent e) -> { // 10. Düğmeye Olay Dinleyici (Action  
        Listener) Ekleme  
            // Bu kısım, düğmeye tıklandığında ne olacağını tanımlar.
```

```
// b.addActionListener(...): 'b' düğmesine bir "aksiyon dinleyicisi" ekler.
// Bu dinleyici, düğmeye tıklandığında (aksiyon gerçekleştiğinde) tetiklenir.
// (ActionEvent e) -> { ... }: Bu bir Lambda ifadesidir. ActionListener arayüzünü
//          kısaca uygulamanın modern bir yoludur.
//          ActionListener'ın tek metodu olan 'actionPerformed' metodunu
//          temsil eder.
//          'e' parametresi, olay hakkındaki bilgileri (hangi düğme tıklandı vb.)
//          taşır,
//          ancak bu basit örnekte kullanılmıyor.
// { ... }: Düğmeye tıklandığında çalışacak olan kod bloğu.
tf.setText("You have just clicked the button!"); // 11. Metin Alanının İçeriğini
Değiştirme
// Düğmeye tıklandığında, 'tf' metin alanının içeriği bu metinle değiştirilir.
}); // Olay dinleyicinin sonu

f.add(b); f.add(tf); // 12. Düğmeyi ve Metin Alanını Pencereye Ekleme
// Hem 'b' düğmesi hem de 'tf' metin alanı 'f' penceresine ekleniyor.

f.setSize(300, 300); // 13. Pencerenin Boyutunu Ayarlama
// Pencerenin genişliği 300, yüksekliği 300 piksel olarak ayarlanıyor.

f.setLayout(null); // 14. Layout Manager'ı Devre Dışı Bırakma
// Otomatik bileşen yerleşimi devre dışı bırakılıyor, manuel konumlandırma
// (setBounds) kullanılıyor.

f.setVisible(true); // 15. Pencereyi Görünür Yapma
// Pencere ve içindekiler ekranda gösteriliyor.
}
}
```

Bu kod, "Button Example" başlıklı bir pencere oluşturur. Pencerede bir metin alanı (JTextField) ve "Click Here" yazan bir düğme (JButton) bulunur. Kullanıcı düğmeye tıkladığında, metin alanındaki yazı "You have just clicked the button!" olarak değişir. Bileşenlerin yerleşimi yine manuel olarak setBounds ile yapılmıştır. İlk örnekten farkı, düğmeye tıkladığında bir eylem gerçekleştirmesidir.

TextArea1

```
import javax.swing.JFrame; // 1. Sadece JFrame Sınıfını İçer Aktarma

// Yalnızca pencere oluşturmak için gereken JFrame sınıfı içer aktarılıyor.

import javax.swing.JTextArea; // 2. Sadece JTextArea Sınıfını İçer Aktarma

// Çok satırlı metin alanı için gereken JTextArea sınıfı içer aktarılıyor.

public class TextArea1 { // 3. Sınıf Tanımı

    TextArea1() { // 4. Kurucu Metot (Constructor)

        // Bu, TextArea1 sınıfının kurucu metodudur.

        // 'new TextArea1()' ile bu sınıftan bir nesne oluşturulduğunda otomatik olarak
        çalışır.

        // Adı sınıf adıyla aynıdır ve geri dönüş tipi yoktur (void bile değildir).

        // GUI (Grafiksel Kullanıcı Arayüzü) oluşturma mantığı bu metodun içine
        yerleştirilmiş.

        JFrame f = new JFrame(); // 5. JFrame Nesnesi Oluşturma

        // Pencere nesnesi 'f' oluşturuluyor. Bu sefer başlık belirtilmemiş.

        JTextArea area = new JTextArea("This is a textArea"); // 6. JTextArea Nesnesi
        Oluşturma

        // Çok satırlı metinleri göstermek ve düzenlemek için kullanılan 'area' adında bir
```

```
// JTextArea nesnesi oluşturuluyor.

// "This is a textArea" metni, metin alanının başlangıç içeriği olarak ayarlanıyor.


area.setBounds(20, 40, 300, 300); // 7. Metin Alanının Konumunu ve Boyutunu
Ayarlama

// 'area' metin alanının pencere ('f') içindeki yeri ve boyutları ayarlanıyor.

// x=20, y=40, genişlik=300, yükseklik=300 piksel.


f.add(area); // 8. Metin Alanını Pencereye Ekleme

// Oluşturulan 'area' metin alanı 'f' penceresine ekleniyor.


f.setSize(400, 400); // 9. Pencerenin Boyutunu Ayarlama

// Pencerenin genişliği 400, yüksekliği 400 piksel olarak ayarlanıyor.


f.setLayout(null); // 10. Layout Manager'ı Devre Dışı Bırakma

// Otomatik bileşen yerleşimi devre dışı bırakılıyor, manuel konumlandırma
kullanılıyor.


f.setVisible(true); // 11. Pencereyi Görünür Yapma

// Pencere ve içindeki metin alanı ekranda gösteriliyor.

} // Kurucu Metodun Sonu


public static void main(String[] args) { // 12. Ana Metot (Main Method)

// Uygulamanın başlangıç noktası.


new TextArea1(); // 13. TextArea1 Nesnesi Oluşturma

// Bu satır, TextArea1 sınıfından yeni bir nesne (örnek) oluşturur.

// Bu nesne oluşturulduğunda, 4. adımda tanımlanan kurucu metot (TextArea1())
otomatik olarak çağrılır.
```

```
// Dolayısıyla, bu satırın çalıştırılması, kurucu metot içindeki GUI oluşturma ve gösterme
```

```
// işlemlerini tetikler. Oluşturulan nesneyi bir değişkene atamaya gerek yoktur,
```

```
// çünkü main metodu içinde daha sonra bu nesneye referans vermemiz gerekmiyor.
```

```
} // Ana Metodun Sonu
```

```
} // Sınıfın Sonu
```

Özetle:

Bu kod, `TextArea1` adında bir sınıf tanımlar. Bu sınıftan bir nesne oluşturulduğunda (ki bu `main` metodu içinde yapılır), sınıfın kurucu metodu çalışır. Kurucu metot, 400x400 piksel boyutunda bir pencere oluşturur, bu pencereye başlangıç metni içeren çok satırlı bir metin alanı (`JTextArea`) ekler, metin alanını manuel olarak konumlandırır ve son olarak pencereyi görünür hale getirir. Önceki örneklerden temel farkı, arayüz oluşturma kodunun `main` metodu yerine sınıfın kurucu metodu (constructor) içinde yer almasıdır.

TextArea2

```
import java.awt.event.*; // 1. Olay (Event) İşleme Paketini İçe Aktarma
```

```
// ActionListener ve(ActionEvent) sınıflarını kullanabilmek için gereklidir.
```

```
import javax.swing.*; // 2. Swing GUI Paketini İçe Aktarma
```

```
// JFrame, JLabel, JTextArea, JButton gibi grafiksel arayüz bileşenlerini içerir.
```

```
// 3. Sınıf Tanımı ve ActionListener Arayüzünü Uygulama
```

```
public class TextArea2 implements ActionListener {
```

```
    // 'implements ActionListener' ifadesi, TextArea2 sınıfının ActionListener arayüzünde
```

```
    // tanımlı olan metotları (bu durumda sadece actionPerformed) uygulayacağını belirtir.
```

```
    // Bu sayede TextArea2 sınıfının nesneleri, olay dinleyicisi olarak kullanılabilir.
```

```
// 4. Sınıf Üyeleri (Instance Variables)
```

JLabel l1, l2; // İki adet etiket (metin göstermek için). Kelime ve karakter sayılarını gösterecekler.

JTextArea area; // Çok satırlı metin alanı. Kullanıcı buraya yazı yazacak.

JButton b; // Düğme. Tıklandığında sayma işlemini tetikleyecek.

// 5. Kurucu Metot (Constructor)

TextArea2() {

JFrame f = new JFrame(); // Pencere nesnesi oluşturuluyor.

// Etiket 1'i oluştur ve konumlandır

l1 = new JLabel(); // Boş bir etiket nesnesi oluşturuluyor. İçeriği daha sonra ayarlanacak.

l1.setBounds(120, 75, 100, 30); // Etiket konumu (x=120, y=75) ve boyutu (genişlik=100, yükseklik=30).

// Etiket 2'yi oluştur ve konumlandır

l2 = new JLabel(); // Boş bir etiket nesnesi oluşturuluyor.

l2.setBounds(200, 75, 100, 30); // İkinci etiket konumu ve boyutu. l1'in sağında yer alacak.

// Metin alanını oluştur ve konumlandır

area = new JTextArea(); // Boş bir metin alanı nesnesi oluşturuluyor.

area.setBounds(130, 100, 200, 200); // Metin alanının konumu ve boyutu.

// Düğmeyi oluştur ve konumlandır

b = new JButton("Count Words"); // Üzerinde "Count Words" yazan bir düğme oluşturuluyor.

b.setBounds(165, 300, 120, 30); // Düğmenin konumu ve boyutu.


```
// 6. Düğmeye Olay Dinleyici Ekleme

b.addActionListener(this);

// 'b' düğmesine bir aksiyon dinleyicisi ekleniyor.

// 'this' anahtar kelimesi: Bu satırın içinde bulunduğu nesneyi, yani o an
oluşturulmakta

// olan TextArea2 nesnesini ifade eder. TextArea2 sınıfı ActionListener arayüzünü
// uyguladığı için, 'this' geçerli bir ActionListener'dır.

// Bu sayede 'b' düğmesine tıklandığında, bu nesnenin (TextArea2 nesnesinin)
// actionPerformed metodu çağrılacaktır.


// 7. Bileşenleri Pencereye Ekleme

f.add(l1); f.add(l2); f.add(area); f.add(b);

// Oluşturulan tüm etiketler, metin alanı ve düğme pencereye ekleniyor.


// 8. Pencere Ayarları

f.setSize(450, 450); // Pencerenin boyutu ayarlanıyor.

f.setLayout(null); // Otomatik yerleşim yöneticisi devre dışı bırakılıyor.

f.setVisible(true); // Pencere görünür hale getiriliyor.

} // Kurucu Metodun Sonu


// 9. actionPerformed Metodu (ActionListener Arayüzünden Gelen Zorunlu Metot)

public void actionPerformed(ActionEvent e) {

// Bu metot, dinleyici olarak eklenen bileşende (yani 'b' düğmesinde) bir
// aksiyon (tıklama) gerçekleştiğinde otomatik olarak çağrılır.

// ActionEvent e: Olay hakkında bilgi taşıyan nesne (hangi düğmenin tıklandığı vb.),
// bu örnekte doğrudan kullanılmıyor.

String text = area.getText(); // 10. Metin Alanındaki Yazıyı Alma
```

// 'area' isimli JTextArea bileşeninin içindeki tüm metni alır ve 'text' değişkenine atar.

String words[] = text.split("\\s+"); // 11. Metni Kelimelere Ayırma

// 'text' değişkenindeki metni boşluk karakterlerine ("\\s+") göre böler.

// "\\s+" bir veya daha fazla boşluk karakteri (boşluk, tab, yeni satır vb.) anlamına gelir.

// Bölme işlemi sonucunda elde edilen kelimeler (String dizisi) 'words' dizisine atanır.

// Not: Eğer metin başında/sonunda boşluk varsa veya kelimeler arasında birden fazla boşluk varsa,

// split metodu genellikle bunları doğru şekilde işler ve boş elemanlar oluşturmaz (regex'e bağlı olarak).

l1.setText("Words: " + words.length); // 12. Kelime Sayısını Etikete Yazma

// 'l1' etiketinin metnini "Words: " ve ardından bulunan kelime sayısı (words dizisinin eleman sayısı) günceller.

l2.setText("Characters: " + text.length()); // 13. Karakter Sayısını Etikete Yazma

// 'l2' etiketinin metnini "Characters: " ve ardından metnin toplam karakter sayısı (boşluklar dahil) günceller.

} // actionPerformed Metodunun Sonu

// 14. Ana Metot (Main Method)

public static void main(String[] args) {

new TextArea2(); // 15. TextArea2 Nesnesi Oluşturma

// TextArea2 sınıfından bir nesne oluşturulur. Bu işlem kurucu metodu (TextArea2()) tetikler

// ve dolayısıyla grafiksel arayüz oluşturulur ve gösterilir. Olay dinleyicisi de ayarlanmış olur.

} // Ana Metodun Sonu

```
} // Sınıfın Sonu
```

Özetle:

Bu kod, bir metin alanı, iki etiket ve "Count Words" etiketli bir düğme içeren bir pencere oluşturur. TextArea2 sınıfı, ActionListener arayüzünü uygular, bu da kendi nesnesinin düğme tıklamalarını dinleyebilmesini sağlar. Düğmeye tıklandığında, actionPerformed metodu çalışır: metin alanındaki yazıyı alır, boşluklara göre kelimelere ayırır, kelime sayısını ve toplam karakter sayısını hesaplar ve bu sayıları iki ayrı etikette gösterir. Bu örnek, olayı işleyecek metodun doğrudan arayüzü oluşturan sınıfın içinde tanımlandığı bir yapı sunar.

GirisEkranı

```
import javax.swing.JButton; import javax.swing.JFrame; import javax.swing.JLabel;
import javax.swing.JPasswordField; // Şifre alanı için gerekli sınıf import
javax.swing.JTextField; import java.awt.event.ActionEvent; import
java.awt.event.ActionListener;
```

```
public class GirisEkranı { // 1. Sınıf Tanımı (GirisEkranı = Login Screen) public static void
main(String[] args) { // 2. Ana Metot (Main Method) // Tüm arayüz oluşturma ve olay
işleme mantığı main metodu içinde yer alıyor.
```

```
    JFrame f = new JFrame("PasswordField2"); // 3. JFrame Oluşturma
    // "PasswordField2" başlıklı bir pencere ('f') oluşturuluyor.
```

```
    JLabel label = new JLabel(); // 4. Sonuç Etiketi Oluşturma
    // Giriş yapıldıktan sonra kullanıcı adı ve şifreyi göstermek
    için kullanılacak
    // boş bir etiket ('label') oluşturuluyor.
    label.setBounds(20, 130, 250, 80); // Etiket konumu ve boyutu
    ayarlanıyor.
```

```
    // Diğer bileşenlerin altına
    yerleştirilmiş.
```

```
    JPasswordField field = new JPasswordField(); // 5.
    JPasswordField Oluşturma
```

```
    // Şifre girişi için özel bir metin alanı ('field')
    oluşturuluyor.
```

```
    // Bu alan, içine yazılan karakterleri genellikle '*' veya '.'
    gibi karakterlerle maskeler.
```

```
    field.setBounds(100, 75, 100, 25); // Şifre alanının konumu ve
    boyutu ayarlanıyor.
```

```
JLabel l1 = new JLabel("Username: "); // 6. Kullanıcı Adı Etiketi
// "Username: " metnini gösteren bir etiket ('l1')
oluşturuluyor.
l1.setBounds(20, 20, 80, 25); // Etiketin konumu ve boyutu.

JLabel l2 = new JLabel("Password: "); // 7. Şifre Etiketi
// "Password: " metnini gösteren bir etiket ('l2')
oluşturuluyor.
l2.setBounds(20, 75, 80, 25); // Etiketin konumu ve boyutu.
Şifre alanının soluna yerleştirilmiş.

JButton b = new JButton("Login"); // 8. Giriş Düğmesi
// Üzerinde "Login" yazan bir düğme ('b') oluşturuluyor.
b.setBounds(100, 120, 80, 25); // Düğmenin konumu ve boyutu.

JTextField text = new JTextField(); // 9. Kullanıcı Adı Metin Alanı
// Kullanıcı adının girileceği standart bir metin alanı ('text')
oluşturuluyor.
text.setBounds(100, 20, 100, 25); // Metin alanının konumu ve boyutu. Kullanıcı adı etiketinin sağına yerleştirilmiş.

// 10. Bileşenleri Pencereye Ekleme
f.add(field); // Şifre alanı
f.add(l1);    // Kullanıcı adı etiketi
f.add(label); // Sonuç etiketi
f.add(l2);    // Şifre etiketi
f.add(b);     // Giriş düğmesi
f.add(text);  // Kullanıcı adı metin alanı
// setLayout(null) kullanıldığı için ekleme sırası genellikle
görsel düzeni etkilemez,
// ancak üst üste binen bileşenlerde son eklenen üstte
kalabilir.

// 11. Pencere Ayarları
f.setSize(400, 400); // Pencerenin boyutu (400x400 piksel).
f.setLayout(null); // Otomatik yerleşim yöneticisi devre dışı.
Manuel konumlandırma kullanılıyor.
f.setVisible(true); // Pencere görünür yapılıyor.
```

```
// 12. Düğmeye Olay Dinleyici Ekleme (Anonim İç Sınıf
Kullanarak)
b.addActionListener(new ActionListener() {
    // new ActionListener() { ... }: Bu yapı, ActionListener
arayüzünü uygulayan
    // isimsiz (anonim) bir iç sınıf oluşturur ve bu sınıfın bir
nesnesini yaratır.
    // Bu nesne doğrudan addActionListener metoduna parametre
olarak verilir.
    // Lambda ifadesine veya ayrı bir sınıf tanımlamaya
alternatiftir.
```

```
// 13. actionPerformed Metodu (Anonim Sınıf İçinde)
public void actionPerformed(ActionEvent e) {
    // Bu metod, 'b' düğmesine tıklandığında çalıştırılacak
kodu içerir.
```

```
String data = "Username: " + text.getText(); // 14.
Kullanıcı Adını Alma
    // 'text' (JTextField) alanındaki metni alır ve
"Username: " metniyle birleştirir.
```

```
data += ", Password: " + new
String(field.getPassword()); // 15. Şifreyi Alma ve Ekleme
    // 'field' (JPasswordField) alanından şifreyi alır.
    // field.getPassword(): Güvenlik nedeniyle şifreyi
String olarak değil,
    // char[] (karakter dizisi) olarak
döndürür.
```

```
    // new String(...): Karakter dizisini yazdırmak veya
birleştirmek için
    // geçici olarak String'e dönüştürür.
    // **Not:** Gerçek uygulamalarda
şifreleri String'e dönüştürmekten
    // kaçınılmalı ve char[] dizisi
kullanıldıktan sonra içeriği
    // temizlenmelidir. Bu örnek sadece
gösterme amaçlıdır.
    // Elde edilen şifre, ", Password: " metniyle birlikte
'data' değişkenine eklenir.
```

```
label.setText(data); // 16. Sonucu Etikete Yazma
```

```

        // Oluşturulan 'data' string'i (kullanıcı adı ve şifreyi
        içeren)
        // başlangıçta boş olan 'label' etiketinin metni olarak
        ayarlanır.
    } // actionPerformed metodunun sonu
}); // Anonim ActionListener tanımının sonu

} // main metodunun sonu

} // GirişEkranı sınıfının sonu

```

Özetle:

Bu kod, "PasswordField2" başlıklı basit bir giriş ekranı arayüzü oluşturur. Kullanıcı adı için normal bir metin alanı (JTextField), şifre için maskeli bir metin alanı (JPasswordField), ilgili etiketler ve bir "Login" düğmesi içerir. Ayrıca, giriş bilgilerini göstermek için başta boş olan bir etiket bulunur. "Login" düğmesine tıklandığında, bir *anonim iç sınıf* tarafından tanımlanan olay dinleyici (ActionListener) tetiklenir. Bu dinleyici, kullanıcı adı ve şifre alanlarındaki verileri alır (şifreyi getPassword() ile alıp String'e çevirir), bu bilgileri birleştirir ve sonuç etiketinde gösterir. Bu örnek, olay dinleyicilerini tanımlamanın başka bir yaygın yolu olan anonim iç sınıfları göstermektedir.

IPFinderApp

import javax.swing.; // Swing GUI bileşenleri için (JFrame, JTextField, JLabel, JButton, JOptionPane) import java.awt.; // AWT bileşenleri için (Layout yöneticileri - FlowLayout, Renk - Color) import java.awt.event.*; // Olay dinleyicileri için (ActionListener,(ActionEvent) import java.net.UnknownHostException; // Ağ hatası (bilinmeyen ana bilgisayar) için istisna sınıfı // java.net.InetAddress sınıfı da dolaylı olarak kullanılır, ancak doğrudan import edilmemiş.

// 1. Sınıf Tanımı: JFrame'den miras alıyor ve ActionListener'ı uyguluyor public class IPFinderApp extends JFrame implements ActionListener { // 'extends JFrame': IPFinderApp sınıfının bir JFrame (pencere) olduğunu belirtir. // Bu sayede pencereye ait metotları (setSize, setLayout, add vb.) doğrudan kullanabiliriz. // 'implements ActionListener': Bu sınıfın ActionListener arayüzünü uyguladığını, yani // actionPerformed metodunu içerdiğini belirtir. Düğme tıklamalarını // bu sınıfın kendisi dinleyecektir.

// 2. GUI Bileşenleri (Instance Variables - Sınıf Üyeleri) private final JTextField tf; // Kullanıcının alan adını (domain)

```
gireceği metin alanı. 'final' olduğu için constructor'da atanmalı.
private final JLabel l;          // Sonucun (IP adresi veya hata)
gösterileceği etiket. 'final'.
private final JButton b;         // IP bulma işlemini tetikleyecek
düğme. 'final'.

// 3. Kurucu Metot (Constructor)
public IPFinderApp() {
    super("IP Finder"); // 4. Üst Sınıfın (JFrame) Constructor'ını
    Çağırma
                                // Pencerenin başlığını "IP Finder" olarak
    ayarlar.

    // 5. Metin Alanını (JTextField) Oluşturma ve Ayarlama
    tf = new JTextField("Enter a domain (e.g. google.com)", 20);
    // İçinde başlangıçta bir ipucu metni olan ve yaklaşık 20
    karakter genişliğinde
    // bir metin alanı oluşturur.
    tf.setForeground(Color.GRAY); // İpucu metninin rengini gri
    yapar (daha iyi bir kullanıcı deneyimi için).
                                // Not: Gerçek placeholder
    işlevselliği için ek kod (FocusListener) gerekebilir.

    // 6. Etiket (JLabel) Oluşturma
    l = new JLabel("Result will be displayed here."); // Başlangıç
    metni olan etiketi oluşturur.

    // 7. Düğmeyi (JButton) Oluşturma ve Dinleyici Ekleme
    b = new JButton("Find IP"); // Üzerinde "Find IP" yazan düğmeyi
    oluşturur.
    b.addActionListener(this); // Bu düğmenin aksiyonlarını
    dinlemesi için mevcut IPFinderApp nesnesini ('this') ekler.
                                // Düğmeye tıklandığında bu sınıfın
    actionPerformed metodu çağrılır.

    // 8. Layout Yöneticisini Ayarlama
    setLayout(new FlowLayout()); // Bileşenlerin soldan sağa doğru
    sıralanmasını ve sığmazsa
                                // alt satıra geçmesini sağlayan
    FlowLayout yöneticisini kullanır.
                                // Önceki örneklerdeki
    'setLayout(null)' yerine daha esnek bir yöntemdir.
```

```

// 9. Bileşenleri Pencereye (Frame) Ekleme
add(tf); // Metin alanını ekler.
add(b); // Düğmeyi ekler.
add(l); // Etiketi ekler.
        // FlowLayout nedeniyle eklenme sırasına göre soldan
sağa dizilirler.

// 10. Pencere Ayarları
setSize(400, 200); // Pencerenin başlangıç boyutunu ayarlar
(genişlik=400, yükseklik=200).
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// Pencerenin kapatma düğmesine (X) tıklandığında uygulamanın
tamamen sonlanmasını sağlar.
// Bu satır olmazsa, pencere kapansa bile program arka planda
çalışmaya devam edebilir.
setVisible(true); // Pencereyi görünür hale getirir.
} // Kurucu Metodun Sonu

// 11. actionPerformed Metodu (ActionListener Arayüzünden Gelen
Zorunlu Metot)
@Override // Bu metodun üst sınıftaki (veya arayüzdeki) bir metodu
ezdiğini belirtir (iyi pratik).
public void actionPerformed(ActionEvent e) {
    // Bu metot, 'b' düğmesine tıklandığında otomatik olarak
çağrılır.

    try { // 12. Hata Yakalama Bloğu (Try-Catch)
        // Ağ işlemleri (InetAddress.getByName) hata verebileceği
için (örn. geçersiz alan adı,
        // internet bağlantısı yok) try-catch bloğu kullanmak
zorunludur.

        String host = tf.getText().trim(); // 13. Metin Alanından
Veri Alma ve Temizleme
                                                // Kullanıcının girdiği
metni alır ve başındaki/sonundaki boşlukları 'trim()' ile temizler.

        // 14. Giriş Doğrulama (Input Validation)
        if (host.isEmpty() || host.equalsIgnoreCase("Enter a domain
(e.g. google.com))) {
            // Eğer temizlenmiş metin boşsa VEYA hala başlangıçtaki

```



```

ipucu metnine eşitse (büyük/küçük harf duyarlı):
        JOptionPane.showMessageDialog(this, // Hata mesajını
gösterecek pencere (bu frame)
                                "Please enter a valid
domain name!", // Gösterilecek mesaj
                                "Error", // Mesaj
kutusunun başlığı

JOptionPane.ERROR_MESSAGE); // Mesaj kutusunun türü (hata ikonu)
        return; // Hata varsa, metodun geri kalanını
çalıştırmadan çıkar.
    }

    // 15. IP Adresini Bulma (Ağ İşlemi)
    String ip =
java.net.InetAddress.getByName(host).getHostAddress();
    // java.net.InetAddress.getByName(host): Verilen 'host' adı
için DNS sorgusu yapar.
    //
InetAddress nesnesi döndürür.
    //
(örn. domain yok) UnknownHostException fırlatır.
    // .getHostAddress(): Bulunan InetAddress nesnesinden IP
adresini String olarak alır.

    l.setText("IP of " + host + " is: " + ip); // 16. Sonucu
Etikete Yazma
                                // Başarılı
olursa, bulunan IP adresini etikette gösterir.

    } catch (UnknownHostException ex) { // 17. Hata Yakalama
        // Eğer try bloğu içindeki InetAddress.getByName(...) bir
UnknownHostException fırlatırsa:
        JOptionPane.showMessageDialog(this,
                                "Invalid domain or no internet
connection!", // Hata mesajı
                                "Error", // Başlık
JOptionPane.ERROR_MESSAGE); //
Tür
    }
} // actionPerformed Metodunun Sonu

```

```
// 18. Ana Metot (Main Method)
public static void main(String[] args) {
    new IPFinderApp(); // 19. Uygulamayı Başlatma
    // IPFinderApp sınıfından bir nesne
    oluşturur. Bu, kurucu metodu (constructor)
    // çağırarak GUI'nin kurulmasını ve
    gösterilmesini tetikler.
} // Ana Metodun Sonu

} // IPFinderApp Sınıfının Sonu
```

Özetle:

Bu kod, Swing kullanarak çalışan bir "IP Bulucu" masaüstü uygulaması oluşturur. Kullanıcı bir alan adı (domain name) girip "Find IP" düğmesine tıkladığında, program Java'nın ağ yeteneklerini kullanarak o alan adının IP adresini bulmaya çalışır.

Önemli Noktalar:

- **Yapı:** Ana sınıf (IPFinderApp) hem pencerenin kendisidir (extends JFrame) hem de düğme olaylarını dinler (implements ActionListener).
- **Ağ İşlemi:** java.net.InetAddress.getByName() metodu ile DNS sorgusu yapılır.
- **Hata Yönetimi:** Ağ işlemlerinin başarısız olabileceği göz önünde bulundurularak try-catch bloğu ile UnknownHostException yakalanır ve kullanıcıya JOptionPane ile hata mesajı gösterilir.
- **Giriş Doğrulama:** Kullanıcının geçerli bir giriş yapıp yapmadığı kontrol edilir.
- **Layout Manager:** Bileşenleri düzenlemek için setLayout(null) yerine FlowLayout kullanılmıştır, bu daha esnek bir arayüz sağlar.
- **Kapanış Davranışı:** setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) ile pencere kapatıldığında uygulamanın düzgün şekilde sonlanması sağlanır.

PasswordField1

```
import javax.swing.JFrame; // Pencere oluşturmak için
import javax.swing.JLabel; // Metin etiketi oluşturmak için
import javax.swing.JPasswordField; // Şifre giriş alanı oluşturmak için

public class PasswordField1 { // 1. Sınıf Tanımı
```

```
public static void main(String[] args) { // 2. Ana Metot (Main Method)

    // Uygulamanın başlangıç noktası. Tüm kod bu metot içinde.

    JFrame f = new JFrame("Password Field Example"); // 3. JFrame Oluşturma
    // "Password Field Example" başlıklı bir pencere ('f') oluşturuluyor.

    JPasswordField passf = new JPasswordField(); // 4. JPasswordField Oluşturma
    // Şifre girişi için kullanılacak özel metin alanı ('passf') oluşturuluyor.
    // Bu alana yazılan karakterler görünmez (genellikle '*' veya '•' ile maskelenir).

    JLabel l1 = new JLabel("Password: "); // 5. JLabel Oluşturma
    // "Password: " metnini gösteren bir etiket ('l1') oluşturuluyor.

    l1.setBounds(20, 100, 80, 30); // 6. Etiket Konumunu ve Boyutunu Ayarlama
    // 'l1' etiketinin pencere içindeki yerini ve boyutlarını ayarlar.
    // x=20 (sol kenardan uzaklık), y=100 (üst kenardan uzaklık),
    // genişlik=80, yükseklik=30 piksel.

    passf.setBounds(100, 100, 100, 30); // 7. Şifre Alanının Konumunu ve Boyutunu
    Ayarlama
    // 'passf' şifre alanının pencere içindeki yerini ve boyutlarını ayarlar.
    // x=100, y=100, genişlik=100, yükseklik=30 piksel.
    // y koordinatının etiketle aynı (100) olması, dikey olarak aynı hizada olduklarını
    gösterir.
    // x koordinatının 100 olması, etiketin bittiği yerden (20 + 80 = 100) başlamasını
    sağlar,
    // yani etiket ve şifre alanı yan yana durur.

    f.add(passf); // 8. Şifre Alanını Pencereye Ekleme
```

```
f.add(l1); // 9. Etiket Pencereye Ekleme

// Bileşenler pencereye ekleniyor.

f.setSize(300, 300); // 10. Pencerenin Boyutunu Ayarlama

// Pencerenin genişliği 300, yüksekliği 300 piksel olarak ayarlanıyor.

f.setLayout(null); // 11. Layout Manager'ı Devre Dışı Bırakma

// Otomatik bileşen yerleşimi kapatılıyor, 'setBounds' ile manuel konumlandırma yapılıyor.

f.setVisible(true); // 12. Pencereyi Görünür Yapma

// Oluşturulan pencere ve içindeki bileşenler ekranda gösteriliyor.

} // Ana Metodun Sonu

} // Sınıfın Sonu
```

Özetle:

Bu kod parçası, Swing kütüphanesini kullanarak çok basit bir arayüz oluşturur. "Password Field Example" başlıklı bir pencere yaratır. Pencerenin içine "Password: " yazan bir etiket ve hemen yanına, kullanıcının şifre girebileceği (ve girilen karakterlerin maskeleneceği) bir `JPasswordField` bileşeni yerleştirir. Bileşenlerin konumları `setBounds` metodu ile manuel olarak belirlenmiştir.

Bu kod, sadece `JPasswordField` bileşeninin nasıl oluşturulup pencereye ekleneceğini gösterir. Girilen şifreyi almak veya kontrol etmek gibi herhangi bir işlevsellik (olay dinleme) içermez.

SplitExample

```
public class SplitExample { // 1. Sınıf Tanımı

    public static void main(String[] args) { // 2. Ana Metot (Main Method)

        // Uygulamanın başlangıç noktası.

        String text = "Hello world! This is a test."; // 3. Örnek Metin (String)
```

```
// Üzerinde işlem yapılacak metin tanımlanıyor.

// Dikkat: "Hello" ve "world!" arasında birden fazla boşluk karakteri var.


// 4. Metni Boşluklara Göre Bölme (split metodu)

String[] words = text.split("\\s+");

// text.split(...): 'text' string'ini belirtilen ayırıcıya göre böler ve

//          sonuçları bir String dizisi (String[]) olarak döndürür.

// "\\s+": Bölme işlemi için kullanılan ayırıcıyı tanımlayan düzenli ifadedir (regular
expression - regex).

//   \\s: Herhangi bir boşluk karakterini temsil eder (boşluk, tab, yeni satır vb.).

//       Java'da String içinde özel karakter olan '\' (ters eğik çizgi) karakterini

//       kullanabilmek için önüne bir tane daha '\' ekleyerek '\\s' şeklinde yazarız.

//   +: Regex'te "bir veya daha fazla" anlamına gelen bir niceleyicidir (quantifier).

// Dolayısıyla "\\s+", "bir veya daha fazla ardışık boşluk karakteri" anlamına gelir.

// Bu ifade sayesinde metin içindeki tek boşluklar da, birden fazla ardışık boşluklar
da

// tek bir ayırıcı olarak kabul edilir. Bu, kelimeler arasında birden fazla boşluk

// olduğunda dahi arada boş elemanlar oluşmasını engeller.

// Kod içindeki yorum "//s'nin yanında + kullanırsak fazladan koyulan boşlukları
saymaz"

// bu durumu doğru bir şekilde açıklar; yani ardışık boşlukları tek bir ayırıcı olarak ele
alır.


// Bölme işlemi sonucunda elde edilen kelimeler 'words' dizisine atanır.

// Bu örnek için 'words' dizisi şu elemanları içerecektir:

// ["Hello", "world!", "This", "is", "a", "test."]


System.out.println("Kelime sayısı: " + words.length); // 5. Kelime Sayısını Yazdırma

// 'words' dizisinin eleman sayısını (yani bulunan kelime sayısını) ekrana yazdırır.
```

```
// Çıktı: Kelime sayısı: 6
```

```
// 6. Kelimeleri Tek Tek Yazdırma (for-each döngüsü)
```

```
for (String word : words) { // 'words' dizisindeki her bir eleman için döngüyü çalıştırır.
```

```
    System.out.println(word); // Döngünün her adımında mevcut 'word' elemanını  
    ekrana yazdırır.
```

```
}
```

```
// Döngünün Çıktısı:
```

```
// Hello
```

```
// world!
```

```
// This
```

```
// is
```

```
// a
```

```
// test.
```

```
} // Ana Metodun Sonu
```

```
} // Sınıfın Sonu
```

Özetle:

Bu kod, Java'da bir metni (String) kelimelere ayırmak için `split()` metodunun nasıl kullanılacağını gösterir. Özellikle `split("\\s+")` ifadesinin, metin içindeki bir veya daha fazla boşluk karakterini tek bir ayırıcı olarak kabul ederek, kelimeler arasındaki düzensiz boşluklara rağmen metni doğru bir şekilde kelimelere ayırmasını sağlar. Kod, önce toplam kelime sayısını, ardından da bulunan her kelimeyi ayrı bir satırda ekrana yazdırır.